RAG Pipeline: File → Chunks → Embeddings → Vector Store → Query → LLM

Python beginner implementation guide | Two major phases

Purpose

This document explains the two core halves of a Retrieval-Augmented Generation (RAG) system. Phase A prepares a knowledge file for search. Phase B accepts a user's question, finds the most relevant stored information, sends that context to an API-based LLM, and prints the answer.

Overall Architecture

```
PHASE A - INDEXING
File of any supported type
↓
File-type detection
↓
Text extraction
↓
Clean / normalize text
↓
Chunking
↓
Embedding model
↓
Vectors + metadata
↓
Vector store

PHASE B - QUERY
User input
↓
Query embedding
↓
Vector similarity search
↓
Top matching chunks
↓
Prompt = instructions + context + question
↓
API LLM
↓
Generated response
↓
Print response
```

Important idea

The LLM does not normally search your document itself. The application performs retrieval first. The LLM receives the selected context and generates a natural-language answer from it.

PHASE A — INDEXING A FILE

1. Read a File of Any Type

There is no single Python function that can meaningfully read every file type. First identify the file type, then use an appropriate parser/loader. Examples include TXT, PDF, DOCX, CSV, XLSX, PPTX, HTML, and JSON.

Recommended approach

```
input file
↓
detect extension / MIME type
↓
choose loader
```

```
■■■ .txt → text reader
■■■ .pdf → PDF text loader
■■■ .docx → DOCX parser
■■■ .csv → CSV reader
■■■ .xlsx → spreadsheet parser
■■■ .pptx → PowerPoint parser
■■■ .html → HTML parser
■■■ .json → JSON parser
↓
standardized Document objects
```

For a RAG project, convert supported formats into a common internal representation such as text plus metadata.

```
Document(
text = "...extracted text...",
metadata = {
"source": "manual.pdf",
"page": 12,
"file_type": "pdf"
}
)
```

Why metadata matters

Metadata lets you later identify the source document and page. It can also support filtering by document, department, customer, or authorized scope.

2. Extract Text

Examples of Python libraries commonly used for RAG document extraction include PyMuPDF or pypdf for PDF, python-docx for DOCX, pandas/openpyxl for tabular files, python-pptx for PowerPoint, BeautifulSoup for HTML, and Python's built-in json module for JSON. RAG frameworks can also provide loaders around these libraries.

Example: TXT

```
from pathlib import Path

text = Path("documents/manual.txt").read_text(
encoding="utf-8"
)
```

Example: PDF

```
import fitz # PyMuPDF

pdf = fitz.open("documents/manual.pdf")

for page_number, page in enumerate(pdf, start=1):
text = page.get_text()
print(page_number, text[:200])
```

The key design principle is: all supported file types should eventually become **text plus metadata**.

3. Clean and Normalize Text

Extracted text may contain extra spaces, repeated line breaks, headers, footers, or formatting artifacts. Clean only what is safe to remove; preserve meaningful structure such as headings and paragraphs.

```
raw text
↓
remove obvious noise
↓
normalize whitespace
↓
preserve meaningful boundaries
↓
clean text
```

4. Split Text into Chunks

An embedding model and vector store work better when a large document is divided into smaller meaningful pieces.

```
Large document
|
+--> Chunk 1
+--> Chunk 2
+--> Chunk 3
+--> Chunk 4
...
```

A common beginner strategy is recursive text splitting with a chunk size and overlap. Start with a reasonable value and tune it based on retrieval quality; there is no universal best chunk size.

Why overlap?

If a sentence or concept crosses a chunk boundary, overlap gives the next chunk some surrounding context.

```
Chunk 1:
"...ICCID is a unique identifier for a SIM..."

Chunk 2:
"...unique identifier for a SIM. It is normally
associated with the physical or eSIM profile..."
```

5. Attach Metadata to Every Chunk

```
Chunk:
"ICCID is ..."

Metadata:
{
"source": "telecom_manual.pdf",
"page": 24,
"chunk_id": "manual-24-03"
}
```

The chunk and metadata should travel together into the vector store.

6. Convert Chunks to Embeddings

An embedding model converts text into a high-dimensional numerical vector representing semantic information.

```
Chunk text
↓
Embedding model
↓
Vector
[0.018, -0.231, 0.772, ...]
```

Every chunk gets its own vector. Later, the user question is embedded using the same compatible embedding model so the vector store can compare them.

```
def embed_text(text):
# call selected embedding model/API
return vector
```

In a real implementation, use the official client or supported RAG framework integration for your chosen embedding model rather than implementing the neural embedding algorithm yourself.

7. Store Embeddings in a Vector Store

The vector store keeps the vector, original chunk text, and metadata together.

```
VectorStore
-----------------------------------------------
ID Vector Text Metadata
-----------------------------------------------
001 [ ... ] Chunk A source/page
002 [ ... ] Chunk B source/page
003 [ ... ] Chunk C source/page
-----------------------------------------------
```

For a beginner prototype, use a local vector store. Options include FAISS, Chroma, Qdrant, or a vector capability exposed through a RAG framework. Choose one rather than installing all of them.

8. Complete Indexing Flow

```
def index_file(file_path):

documents = load_file(file_path)

documents = clean(documents)

chunks = split_into_chunks(documents)

vectors = embed(chunks)

vector_store.add(
vectors=vectors,
texts=chunks,
metadata=metadata
)

vector_store.persist()
```

This is pseudocode showing the sequence. The exact code depends on the selected libraries and providers.

Indexing checkpoint

```
[ ] File was read
[ ] Text/pages were extracted
[ ] Chunks were created
[ ] Embeddings were generated
[ ] Vectors + text + metadata were stored
[ ] A known test query can retrieve a known chunk
```

PHASE B — USER QUERY → RETRIEVAL → LLM

9. Take User Input

The terminal version can start with Python's built-in input() function.

```
question = input("You: ")

if question.lower() == "exit":
break
```

```
You: What is ICCID?
```

10. Convert the User Question to an Embedding

Use the same compatible embedding model used for the document chunks. The question becomes a query vector.

```
User question
"What is ICCID?"
↓
Same embedding model
↓
Query vector
[0.021, -0.218, 0.781, ...]
```

The document vectors and query vector must live in the same embedding space for meaningful similarity search.

11. Search the Vector Store

Send the query vector to the vector store and request the most similar stored vectors.

```
query_vector
↓
vector_store.search(
query_vector,
top_k=3
)
↓
Chunk A
Chunk F
Chunk C
```

The vector store normally calculates a similarity or distance score. Depending on the implementation, this can be cosine similarity, inner product, or another metric.

12. Match the Query with Stored Embeddings

```
Query vector
|
v
+---------------+
| Vector Store |
+---------------+
/ | \
/ | \
Chunk A Chunk B Chunk C
score .91 score .42 score .88
|
+--> select top matching chunks
```

For a question such as **What is ICCID?**, the best matches should be chunks discussing ICCID even when the wording is not identical.

Do not interpret a similarity score as a percentage of correctness. Its meaning depends on the embedding model and distance metric.

13. Retrieve the Actual Text

The vector is used for matching. The LLM normally needs the original text represented by the matching vectors.

```
Search result
|
+--> similarity score
+--> chunk text
+--> metadata
|
v
context for LLM
```

14. Build the LLM Prompt

Combine a grounding instruction, retrieved context, and the user's question.

```
SYSTEM / INSTRUCTION
Answer using the supplied context.
If the answer is not present in the context,
say that it was not found.

CONTEXT
[Chunk 1]
[Chunk 2]
[Chunk 3]

USER QUESTION
What is ICCID?
```

15. Connect Python to an API LLM

Choose an API LLM provider and install its official Python SDK or a supported RAG-framework integration. Keep the API key in an environment variable or approved secret manager.

```
# macOS/Linux example
export LLM_API_KEY="your-api-key"

# Windows PowerShell example
$env:LLM_API_KEY="your-api-key"
```

```
import os

api_key = os.environ["LLM_API_KEY"]
```

The exact SDK initialization and model name depend on the provider. Keep provider-specific code inside a small module so the rest of the RAG application stays portable.

16. Send Query + Retrieved Context to the LLM

```
context = "\n\n".join(
result.text for result in results
)

prompt = f"""
Answer using only the supplied context.

Context:
{context}

Question:
{question}
"""

response = llm.generate(prompt)
```

In production, use the provider's structured chat/messages API when appropriate rather than assuming every model exposes the same interface.

17. Generate and Receive the Response

```
API request
|
v
LLM processes:
```

```
- instructions
- retrieved context
- user question
|
v
Generated answer
|
v
Python receives response
```

The response may include generated text plus usage or other metadata. Extract the text using the response format defined by the SDK.

18. Print the Response

```
answer = response_text

print("\nBot:", answer)
```

For debugging, optionally print the retrieved source names/pages before the final answer. A later UI can display these as source citations.

19. Complete Query-Time Flow

```
while True:

question = input("You: ")

if question.lower() == "exit":
break

query_vector = embed(question)

results = vector_store.search(
query_vector,
top_k=3
)

context = build_context(results)

prompt = build_prompt(
question,
context
)

response = llm.generate(prompt)

print("Bot:", response.text)
```

20. Complete RAG Architecture

```
INDEXING TIME
-------------
File
↓
Loader
↓
Text + metadata
↓
Chunker
↓
Chunks
↓
Embedding model
↓
Vectors
↓
Local Vector Store
↑
|
| retrieval
```

```
|
QUERY TIME
----------
User question
↓
Query embedding
↓
Vector search
↓
Top-K chunks
↓
Context builder
↓
Prompt
↓
API LLM
↓
Response
↓
Terminal
```

21. Example: 'What Is ICCID?'

```
Chunk 17
Source: telecom_basics.pdf
Page: 12

"ICCID stands for Integrated Circuit Card
Identifier. It is a unique identifier associated
with a SIM/eSIM profile or card."
```

```
You: What is ICCID?

"What is ICCID?"
↓
query embedding
↓
vector search
↓
Chunk 17
↓
context
↓
LLM
↓
"ICCID stands for Integrated Circuit Card
Identifier..."
```

22. Important Rule: Retrieval Before Generation

```
Question
↓
Embedding
↓
Vector retrieval
↓
Relevant document context
↓
LLM
↓
Answer
```

If retrieval returns the wrong chunk, debug retrieval first. If retrieval is correct but the answer is poor, investigate the prompt, context size, model, or generation settings.

23. Recommended Python Modules

```
src/
|
+-- main.py
+-- config.py
```

```
+-- loaders.py
+-- chunker.py
+-- embeddings.py
+-- vector_store.py
+-- retriever.py
+-- prompt.py
+-- llm_client.py
+-- chat.py
```

The names are illustrative. The separation makes it easier to replace a loader, embedding provider, vector store, or LLM without rewriting the entire application.

24. Security Notes

Use a non-confidential test document first. Before sending DSP documentation or sensitive telecom information to an external API, follow your organization's security and data-handling requirements. Never hard-code API keys or commit secret files. In production, authorization must determine which documents a user can retrieve; the LLM should not enforce access control.

25. Final Checklist

```
INDEXING
[ ] File type identified
[ ] Correct loader/parser selected
[ ] Text extracted
[ ] Metadata preserved
[ ] Text cleaned
[ ] Text chunked
[ ] Embeddings generated
[ ] Vectors + text + metadata stored

QUERY
[ ] User input read
[ ] Query embedding generated
[ ] Vector similarity search performed
[ ] Top-K chunks retrieved
[ ] Original chunk text extracted
[ ] Grounded prompt built
[ ] API LLM connected
[ ] Context + question sent to LLM
[ ] Response received
[ ] Answer printed
```

26. Recommended Beginner Implementation Order

```
1. Read company.txt
2. Print extracted text
3. Split into chunks
4. Print chunks
5. Generate embeddings
6. Store embeddings + text + metadata
7. Test vector search manually
8. Take terminal question
9. Embed question
10. Retrieve top 3 chunks
11. Print retrieved chunks
12. Connect API LLM
13. Send retrieved context + question
14. Print LLM response
15. Test questions inside and outside the document
```

Once this works with the test document, the same pipeline can be adapted to DSP documents. The next additions would be multiple DSP file types, source citations, access-controlled document filtering, the DSP UI, and eventually a private/local LLM if required.